PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas
Arquitectura de Software

Laboratorio 1

Implementación de un monolito con patrones MVC y DAO

personapi-dotnet — Documento de investigación, diseño y planeación

Integrantes
Integrante 1 — Persistencia: [nombre]
Integrante 2 — Servicios REST: [nombre]
Integrante 3 — Interfaz MVC: Samuel [apellido]

Profesor: [nombre]
Repositorio: github.com/sara-munoz-5/personapi-dotnet
Bogotá D.C., octubre de 2026

Tabla de contenido

Si el índice aparece vacío, en Word haga clic derecho sobre él y elija «Actualizar campos».

1. Introducción

1.1 Objetivo

Construir una sola aplicación ASP.NET Core MVC, llamada personapi-dotnet, que exponga vistas web y endpoints REST para el CRUD (crear, consultar, actualizar y eliminar) de cuatro tablas: profesion, persona, estudios y telefono. El acceso a datos se hace mediante interfaces y repositorios (patrón DAO) sobre Entity Framework Core y SQL Server, y la API se documenta con Swagger (OpenAPI 3).

1.2 Alcance y entregables

El enunciado del laboratorio exige dos entregables:

- **URL del TAG del repositorio git**, que debe contener el README (configuración, pasos para configurar el ambiente, compilación y despliegue), el script DDL, el script DML y el código fuente.
- **Documento** con portada, marco conceptual, diseño, procedimiento, conclusiones y lecciones aprendidas, y referencias.

1.3 Repositorio del equipo

El repositorio público del equipo es github.com/sara-munoz-5/personapi-dotnet. Todo el trabajo se integra allí y la entrega final es la URL del TAG.

1.4 Decisiones de versión

El enunciado pide versiones distintas en la sección Stack (.NET 7, SQL Server 2022) y en el procedimiento (.NET 6, SQL Server 2019 Express). El equipo decidió usar la versión estable más reciente de cada herramienta, con corte al 20 de octubre de 2026.

Tabla 1. *Versiones del enunciado frente a las versiones elegidas por el equipo*

Tema	Sección Stack	Procedimiento	Decisión del equipo
Framework	.NET 7	.NET 6.0	.NET 10 LTS (soporte hasta el 14 de noviembre de 2028)
Motor de base de datos	SQL Server 2022	SQL Server 2019 Express	SQL Server 2025 Express (instancia SQLEXPRESS)
Cliente de base de datos	—	SSMS 18	SSMS 22
IDE	—	Visual Studio Community 2022	Visual Studio Community 2026
Paquetes EF Core	—	Sin versión	10.0.x
Swagger	Swagger 3	—	Swashbuckle.AspNetCore 10.x, documento OpenAPI 3.0

No se usa .NET 11 porque sigue en release candidate; .NET 10 es la versión estable LTS más reciente. .NET 6 y 7 perdieron soporte en 2024, y .NET 8 y 9 lo pierden el 10 de noviembre de 2026. Los pasos del procedimiento siguen siendo válidos: la plantilla MVC, Scaffold-DbContext y la cadena de conexión funcionan igual. **Este cambio debe confirmarse con el profesor antes de la entrega.**

2. Marco conceptual

El laboratorio combina un estilo de arquitectura (monolito), un patrón de presentación (MVC), un patrón de acceso a datos (DAO) y un estilo de integración (REST documentado con OpenAPI).

2.1 Arquitectura monolítica

Un monolito es una aplicación que se compila y despliega como una sola unidad: un proceso, un artefacto y una base de datos compartida. En personapi-dotnet las vistas, la API REST, la lógica y el acceso a datos viven en el mismo proyecto.

Tabla 2. *Ventajas y desventajas del monolito*

Ventajas	Desventajas
Despliegue simple: un solo artefacto	Escala completo o nada
Depuración en un solo proceso	Un cambio obliga a redesplegar toda la aplicación
Sin latencia de red entre módulos	El acoplamiento crece si no se respetan las capas

2.2 Monolito en capas

Que la aplicación se despliegue como una unidad no significa que todo esté mezclado. El código se organiza en capas lógicas: presentación (controladores y vistas), acceso a datos (interfaces y repositorios) y persistencia (entidades y DbContext). Cada capa solo conoce a la de abajo. Las capas son una separación lógica; si se desplegaran en máquinas distintas se llamarían niveles (tiers).

2.3 Patrón MVC (Modelo-Vista-Controlador)

MVC separa la aplicación en tres responsabilidades:

- **Modelo:** representa los datos; aquí, las entidades Persona, Profesion, Estudio y Telefono.
- **Vista:** muestra la información al usuario mediante archivos Razor (.cshtml).
- **Controlador:** recibe la petición HTTP, pide los datos al modelo y decide qué vista devolver.

En ASP.NET Core, el enrutamiento por convención `{controller=Home}/{action=Index}/{id?}` conecta la URL con el método del controlador: /Persona/Edit/123 ejecuta la acción Edit de PersonaController con id = 123.

2.4 Patrón DAO (Data Access Object) y repositorio

El DAO aísla el acceso a datos detrás de una interfaz, de modo que el controlador no sabe si los datos vienen de SQL Server, de un archivo o de un servicio. En .NET se implementa como repositorio: una interfaz por entidad (por ejemplo IPersonaRepository) con las operaciones CRUD, y una clase que la implementa usando EF Core. El controlador recibe la interfaz por inyección de dependencias, lo que permite cambiar la implementación o probar con dobles sin tocar la capa de presentación.

2.5 Inyección de dependencias

ASP.NET Core trae un contenedor de dependencias. En Program.cs se registra qué clase implementa cada interfaz, y el framework entrega la implementación al constructor de cada controlador. El DbContext se

registra con vida Scoped (una instancia por petición) y los repositorios deben tener la misma vida para compartirlo.

2.6 REST

REST es un estilo de arquitectura para APIs sobre HTTP definido por Roy Fielding. Cada recurso tiene una URL (/api/personas/{cc}) y los verbos HTTP expresan la operación: GET consulta, POST crea, PUT actualiza y DELETE elimina. Las respuestas usan JSON y códigos de estado (200, 201, 204, 400, 404, 409). La API no guarda estado de sesión entre peticiones.

2.7 OpenAPI 3 y Swagger

OpenAPI es la especificación estándar para describir una API REST en JSON o YAML; «Swagger 3» se refiere a OpenAPI 3.0. La librería Swashbuckle.AspNetCore genera esa especificación a partir de los controladores y publica Swagger UI, una página web para ver y probar los endpoints.

2.8 ORM y Entity Framework Core

Un ORM (mapeo objeto-relacional) traduce entre tablas y clases. EF Core es el ORM de .NET: el DbContext representa la sesión con la base de datos, cada DbSet representa una tabla y las consultas LINQ se traducen a SQL.

2.9 Database-first y scaffolding

En este laboratorio la base de datos se crea primero y el código se genera a partir de ella. El comando Scaffold-DbContext lee el esquema de persona_db y genera una clase por tabla y el contexto. El enfoque contrario es code-first, en el que las clases generan la base mediante migraciones; los tutoriales de dev.to de la guía del curso usan code-first, por lo que sirven como guía conceptual pero no se siguen al pie de la letra.

3. Herramientas y tecnologías

Todo el ambiente corre en Windows: Visual Studio Community y SQL Server Express no tienen versión para macOS. Quien tenga Mac necesita una máquina virtual Windows o trabajar en el equipo de un compañero.

Tabla 3. *Herramientas, versiones y uso*

Herramienta	Versión	Uso en el laboratorio
Git + GitHub	—	Control de versiones, repositorio del equipo y TAG de entrega
SQL Server Express	2025, instalación Básica	Motor de base de datos; instancia localhost\SQLEXPRESS; máximo 50 GB por base
SQL Server Management Studio (SSMS)	22	Crear persona_db, ejecutar DDL y DML, asignar el dueño sa
Visual Studio Community	2026	IDE: crear el proyecto, compilar, ejecutar y depurar
.NET SDK	10.0 (LTS)	Framework destino del proyecto
ASP.NET Core MVC	10.0	Controladores, vistas Razor y API REST en una sola aplicación
Microsoft.EntityFrameworkCore	10.0.x	ORM base
Microsoft.EntityFrameworkCore.SqlServer	10.0.x	Proveedor de EF Core para SQL Server
Microsoft.EntityFrameworkCore.Tools	10.0.x	Habilita Scaffold-DbContext en la Consola del Administrador de paquetes
Swashbuckle.AspNetCore	10.x	Genera la especificación OpenAPI 3.0 y Swagger UI
Bootstrap	5 (incluido en la plantilla)	Estilos de las vistas CRUD

3.1 Cargas de trabajo de Visual Studio

En el instalador de Visual Studio 2026 se marcan las cargas y componentes que pide el enunciado: Desarrollo de ASP.NET y web, Almacenamiento y procesamiento de datos, Plantillas de proyecto y elementos de .NET Framework, y Características avanzadas de ASP.NET. Si alguno ya no aparece con ese nombre, las dos primeras cargas bastan para este laboratorio.

3.2 Swagger en .NET 10

Desde .NET 9 las plantillas ya no incluyen Swashbuckle, y en .NET 10 el generador nativo (Microsoft.AspNetCore.OpenApi) produce OpenAPI 3.1 por defecto. Para cumplir con «Swagger 3» se instala Swashbuckle.AspNetCore 10.x, que genera OpenAPI 3.0 y publica Swagger UI en /swagger.

4. Diseño

4.1 Arquitectura del monolito

Toda la aplicación es un proceso, pero por dentro una petición baja por capas y cada capa solo conoce a la de abajo. Las vistas y la API comparten la misma capa de acceso a datos.

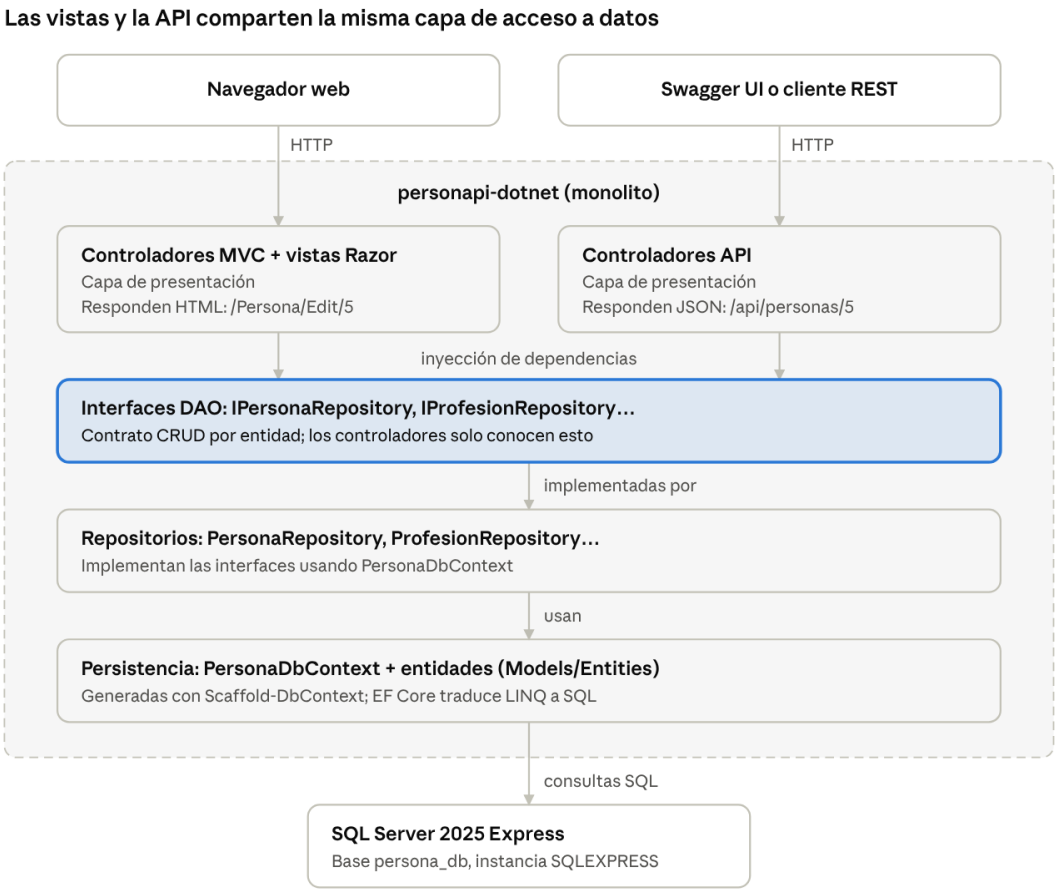


Figura 1. *Arquitectura en capas del monolito personapi-dotnet*

Ejemplo, editar una persona: el navegador envía el formulario a PersonaController.Edit; el controlador llama a IPersonaRepository; PersonaRepository usa PersonaDbContext para actualizar la fila en persona_db. Si la misma edición llega por PUT /api/personas/{cc}, el controlador API llama exactamente al mismo método del repositorio.

4.2 Estructura de carpetas del proyecto

El repositorio tiene la solución, los scripts y la documentación en la raíz, y un único proyecto donde cada carpeta corresponde a una capa de la arquitectura. Es la estructura que sigue el procedimiento del laboratorio.

```
personapi-dotnet/                    ← raíz del repositorio
├── README.md
├── .gitignore
```

```
├── personapi-dotnet.sln
├── scripts/
│   ├── 01_ddl.sql                    ← base y tablas
│   └── 02_dml.sql                    ← datos de prueba
├── docs/
│   └── capturas/                     ← Swagger, vistas, SSMS
└── personapi-dotnet/                 ← proyecto MVC (.NET 10)
    ├── personapi-dotnet.csproj
    ├── Program.cs
    ├── appsettings.json
    ├── appsettings.Development.json
    ├── Properties/
    │   └── launchSettings.json
    ├── Controllers/
    │   ├── HomeController.cs
    │   ├── PersonaController.cs
    │   ├── ProfesionController.cs
    │   ├── EstudioController.cs
    │   ├── TelefonoController.cs
    │   └── Api/
    │       ├── PersonaApiController.cs
    │       ├── ProfesionApiController.cs
    │       ├── EstudioApiController.cs
    │       └── TelefonoApiController.cs
    ├── Interfaces/
    │   ├── IPersonaRepository.cs
    │   ├── IProfesionRepository.cs
    │   ├── IEstudioRepository.cs
    │   └── ITelefonoRepository.cs
    ├── Repository/
    │   ├── PersonaRepository.cs
    │   ├── ProfesionRepository.cs
    │   ├── EstudioRepository.cs
    │   └── TelefonoRepository.cs
    ├── Models/
    │   ├── ErrorViewModel.cs
    │   └── Entities/                 ← generado por Scaffold-DbContext
    │       ├── PersonaDbContext.cs
    │       ├── Persona.cs
    │       ├── Profesion.cs
    │       ├── Estudio.cs
    │       └── Telefono.cs
    ├── ViewModels/
    │   ├── EstudioFormViewModel.cs
    │   └── TelefonoFormViewModel.cs
    ├── Views/
    │   ├── _ViewImports.cshtml
    │   ├── _ViewStart.cshtml
    │   ├── Home/Index.cshtml
    │   ├── Persona/                  ← Index, Details, Create, Edit, Delete
    │   ├── Profesion/                ← mismas 5 vistas
    │   ├── Estudio/                  ← mismas 5 vistas
    │   ├── Telefono/                 ← mismas 5 vistas
    │   └── Shared/
```

```
    │       ├── _Layout.cshtml         ← menú principal
    │       └── Error.cshtml
    └── wwwroot/
        ├── css/
        ├── js/
        └── lib/                      ← Bootstrap y jQuery
```

Figura 2. *Estructura de carpetas de personapi-dotnet*

Tabla 4. *Carpetas, capa a la que pertenecen y responsable*

Carpeta	Capa	Responsable
scripts/	Base de datos (DDL y DML)	Integrante 1
Models/Entities/	Persistencia (scaffolding)	Integrante 1
Interfaces/	Acceso a datos: contratos DAO	Integrante 1 (firmas acordadas por los tres)
Repository/	Acceso a datos: implementación	Integrante 1
Controllers/Api/	Presentación: servicios REST	Integrante 2
Controllers/, ViewModels/, Views/	Presentación: interfaz MVC	Integrante 3
docs/, README.md	Documentación	Los tres

Convenciones de nombres

Tabla 5. *Convenciones de nombres*

Elemento	Convención	Ejemplo
Entidad	Singular, PascalCase, como la genera el scaffolding	Persona, Estudio
Interfaz DAO	I + Entidad + Repository	IPersonaRepository
Repositorio	Entidad + Repository	PersonaRepository
Controlador MVC	Entidad + Controller	PersonaController
Controlador API	Entidad + ApiController, con ruta explícita	PersonaApiController → /api/personas
Ruta API	/api/ + recurso en plural, minúsculas	/api/personas
Carpeta de vistas	Igual al controlador sin «Controller»	Views/Persona/
Scripts SQL	Prefijo numérico de orden de ejecución	01_ddl.sql, 02_dml.sql

Los controladores API van en Controllers/Api para evitar conflictos de nombres con los controladores MVC de la misma entidad y para dejar claro qué responde HTML y qué responde JSON.

4.3 Estructura según la industria

En la industria, un monolito .NET de este tipo suele organizarse como una solución con varios proyectos (Clean Architecture). Sigue siendo un monolito porque se despliega una sola aplicación web, pero cada capa es un proyecto que el compilador obliga a respetar.

Tabla 6. *Proyectos de referencia revisados*

Referencia	Qué es	Estructura	.NET
eShopOnWeb	Aplicación de referencia de Microsoft, hoy mantenida por NimblePros; monolito con vistas MVC y API	src/ con ApplicationCore, Infrastructure, Web y PublicApi; tests/ con pruebas unitarias, de integración y funcionales	10.0
ardalis/CleanArchitecture	Plantilla de Steve Smith para iniciar soluciones ASP.NET Core	src/ con Core, UseCases, Infrastructure y Web; tests/ con pruebas unitarias, de integración y funcionales	10.0
Guía de arquitecturas web de Microsoft	Libro oficial de arquitectura web .NET	Describe la evolución: un proyecto con carpetas, luego N capas, luego Clean Architecture	—

Las tres referencias coinciden: separan src/ y tests/; tienen un proyecto núcleo con entidades e interfaces que no depende de nada; un proyecto de infraestructura con el DbContext y los repositorios; y un proyecto web con controladores, vistas y Program.cs, donde se conectan interfaces e implementaciones. También fijan el SDK con global.json y centralizan las versiones NuGet en Directory.Packages.props.

```
personapi-dotnet/                  ← raíz del repositorio
├── README.md
├── .gitignore
├── global.json                    ← fija el SDK de .NET 10
├── Directory.Packages.props       ← versiones NuGet
├── personapi-dotnet.slnx          ← solución (5 proyectos)
├── docs/
├── scripts/
│   ├── 01_ddl.sql
│   └── 02_dml.sql
├── src/
│   ├── PersonApi.Core/            ← núcleo, sin dependencias
│   │   ├── Entities/              ← Persona, Profesion, Estudio, Telefono
│   │   ├── Interfaces/            ← contratos DAO
│   │   └── Exceptions/
│   ├── PersonApi.Infrastructure/  ← depende de Core
│   │   ├── Data/
│   │   │   ├── PersonaDbContext.cs
│   │   │   └── Config/            ← mapeo por tabla
│   │   ├── Repositories/          ← PersonaRepository…
│   │   └── DependencyInjection.cs ← registra dependencias
│   └── personapi-dotnet/          ← web MVC + API
│       ├── Controllers/
│       │   └── Api/
│       ├── ViewModels/
│       ├── Views/
│       ├── wwwroot/
│       ├── appsettings.json
│       └── Program.cs
└── tests/
    ├── PersonApi.UnitTests/       ← pruebas unitarias
    └── PersonApi.IntegrationTests/ ← pruebas con SQL Server
```

Figura 3. *Estructura de personapi-dotnet al estilo de la industria*

Tabla 7. *Comparación de las dos estructuras*

Aspecto	Un solo proyecto (procedimiento del lab)	Varios proyectos (industria)
Capas	Carpetas dentro de personapi-dotnet	Proyectos separados: Core, Infrastructure, Web
Quién impide saltarse capas	La disciplina del equipo	El compilador: Core no puede usar EF Core
Entidades	Models/Entities, generadas en su sitio	Core/Entities; se generan en Infrastructure y se mueven a Core
Scaffold-DbContext	Comando del enunciado tal cual	Requiere -Project, -StartupProject y mover archivos
Pruebas	No contempladas	Proyectos de pruebas desde el inicio
Esfuerzo para este laboratorio	Bajo	Medio
Despliegue	Una aplicación web	Una aplicación web (sigue siendo monolito)

Decisión: el equipo usa la estructura de un solo proyecto (sección 4.2) porque el procedimiento del laboratorio la exige y ya separa las capas por carpetas. La estructura de la industria queda documentada como referencia y como mejora posible.

4.4 Modelo de datos

La base persona_db tiene cuatro tablas; estudios es la tabla puente que relaciona muchas personas con muchas profesiones. El modelo del enunciado viene de MySQL Workbench, así que sus tipos se traducen a SQL Server.

Tabla 8. *Traducción de tipos de MySQL a SQL Server*

En el modelo (MySQL)	En SQL Server	Motivo
INT(6), INT(15), INT(3)	INT	SQL Server no usa ancho de visualización
VARCHAR(n)	VARCHAR(n)	Igual
TEXT	VARCHAR(MAX)	TEXT está obsoleto en SQL Server; EF Core lo mapea igual a string
ENUM('M','F')	CHAR(1) con CHECK (genero IN ('M','F'))	SQL Server no tiene ENUM
DATE	DATE	Igual

Tabla 9. *Diccionario de datos de persona_db*

Tabla	Columna	Tipo	Nulo	Llave
profesion	id	INT	No	PK
profesion	nom	VARCHAR(90)	No	
profesion	des	VARCHAR(MAX)	Sí	
persona	cc	INT	No	PK

Tabla	Columna	Tipo	Nulo	Llave
persona	nombre	VARCHAR(45)	No	
persona	apellido	VARCHAR(45)	No	
persona	genero	CHAR(1), CHECK M o F	No	
persona	edad	INT	Sí	
estudios	id_prof	INT	No	PK compuesta, FK a profesion.id
estudios	cc_per	INT	No	PK compuesta, FK a persona.cc
estudios	fecha	DATE	Sí	
estudios	univer	VARCHAR(50)	Sí	
telefono	num	VARCHAR(15)	No	PK
telefono	oper	VARCHAR(45)	No	
telefono	duenio	INT	No	FK a persona.cc

Tabla 10. *Relaciones del modelo*

Relación	Cardinalidad	Tipo en el diagrama	Al borrar el padre
profesion → estudios	1 a muchos	Identificadora (línea continua)	CASCADE
persona → estudios	1 a muchos	Identificadora (línea continua)	CASCADE
persona → telefono	1 a muchos	No identificadora (línea punteada)	CASCADE

Decisiones a validar en equipo

- **Llaves sin IDENTITY:** el modelo no marca autoincremento, y cc y num son datos reales, así que el usuario las digita.
- **CASCADE en las tres FK:** borrar una persona borra sus estudios y teléfonos; sin esto el DELETE falla por la restricción. La alternativa es borrar los hijos desde el repositorio.
- **INT para cc:** respeta el modelo, pero su máximo es 2.147.483.647 y una cédula mayor no cabe. Si el profesor lo acepta, BIGINT es más seguro.

Resultado esperado del scaffolding

Scaffold-DbContext crea en Models/Entities las clases Profesion, Persona, Estudio y Telefono, y el contexto PersonaDbContext. Las columnas quedan en PascalCase (cc_per pasa a CcPer) y las FK generan propiedades de navegación (Estudio.CcPerNavigation, Estudio.IdProfNavigation, Telefono.DuenioNavigation). El script DML debe insertar primero profesion y persona, y después estudios y telefono, para respetar las FK.

4.5 Endpoints REST

Cada entidad tiene dos puertas de entrada que comparten el mismo repositorio: un controlador API que responde JSON en /api/… y un controlador MVC que devuelve vistas HTML. En total son 20 endpoints REST y 20 vistas.

Tabla 11. *Endpoints REST documentados en /swagger*

Recurso	GET todos	GET uno	POST crear	PUT actualizar	DELETE eliminar
Personas	/api/personas	/api/personas/{cc}	/api/personas	/api/personas/{cc}	/api/personas/{cc}
Profesiones	/api/ profesiones	/api/profesiones/{id}	/api/ profesiones	/api/profesiones/{id}	/api/profesiones/{id}
Estudios	/api/estudios	/api/estudios/ {idProf}/{ccPer}	/api/estudios	/api/estudios/ {idProf}/{ccPer}	/api/estudios/ {idProf}/{ccPer}
Teléfonos	/api/telefonos	/api/telefonos/{num}	/api/telefonos	/api/telefonos/{num}	/api/telefonos/{num}

Tabla 12. *Códigos de respuesta de la API*

Caso	Código HTTP
Consulta exitosa	200 OK
Creación exitosa	201 Created, con la ubicación del nuevo recurso
Actualización o eliminación exitosa	204 No Content
Datos inválidos o id de la URL distinto al del cuerpo	400 Bad Request
Registro no existe	404 Not Found
Llave primaria repetida	409 Conflict

4.6 Vistas MVC

Tabla 13. *Vistas por controlador MVC*

Controlador	Index	Details	Create	Edit	Delete
Persona	Lista de personas	Datos, estudios y teléfonos	Formulario	Formulario con cc bloqueado	Confirmación
Profesion	Lista	Detalle	Formulario	Formulario	Confirmación
Estudio	Lista con nombre de persona y profesión	Detalle	Formulario con listas de persona y profesión	Edita fecha y universidad	Confirmación
Telefono	Lista con nombre del dueño	Detalle	Formulario con lista de dueño	Formulario	Confirmación

Las rutas siguen la convención /Persona/Edit/{id}. En Estudio, las acciones Details, Edit y Delete reciben idProf y ccPer. El menú del layout compartido enlaza las cuatro listas y Swagger.

5. Organización del equipo

El trabajo se reparte por capa: el integrante 1 construye la persistencia, el 2 los servicios REST y el 3 la interfaz MVC. La arquitectura, las pruebas finales y las conclusiones se hacen entre los tres.

5.1 Reparto de responsabilidades

Tabla 14. *Reparto de responsabilidades*

Responsable	Bloque	Qué entrega	Carpetas
Integrante 1	Persistencia	Scripts DDL y DML, base persona_db, scaffolding (entidades y PersonaDbContext), cadena de conexión, interfaces DAO, repositorios y su registro en Program.cs	scripts/, Models/Entities/, Interfaces/, Repository/
Integrante 2	Servicios REST	4 controladores API con los 20 endpoints, códigos HTTP, Swashbuckle y Swagger UI	Controllers/Api/
Integrante 3 (Samuel)	Interfaz MVC	4 controladores MVC, 20 vistas Razor, listas desplegables, validaciones y menú del layout	Controllers/, ViewModels/, Views/, wwwroot/
Los tres	Colaborativo	Arquitectura y firmas de las interfaces, pruebas finales, conclusiones y lecciones aprendidas, README y TAG	docs/, README.md

5.2 Orden de trabajo

1. Los tres acuerdan la arquitectura y las firmas de las interfaces DAO: métodos, parámetros y tipos de retorno. Es el contrato que une las tres partes.
2. El integrante 1 sube a main la base: proyecto, scaffolding, interfaces y conexión. Los integrantes 2 y 3 parten de ahí.
3. Los integrantes 2 y 3 trabajan en paralelo sobre las mismas interfaces. Los controladores MVC no llaman a la API: usan los repositorios igual que ella.
4. El integrante 1 termina los repositorios; mientras tanto, 2 y 3 prueban con los datos del DML.
5. Los tres integran, hacen las pruebas finales, escriben las conclusiones y crean el TAG.

5.3 Firmas DAO a acordar

Cada interfaz DAO necesita cinco operaciones, todas asíncronas (devuelven Task) para no bloquear el servidor mientras espera a SQL Server.

Tabla 15. *Propuesta de operaciones de las interfaces DAO*

Operación	Qué recibe	Qué devuelve	La usa
Listar todos	Nada	Lista de la entidad, con sus relaciones cuando aplica	Index (MVC) y GET todos (API)
Buscar por llave	cc, id, num, o (idProf, ccPer) en Estudio	La entidad o nulo si no existe	Details, Edit, Delete y GET uno
Crear	La entidad	Nada; lanza error si la llave está repetida	Create (MVC) y POST (API)

Operación	Qué recibe	Qué devuelve	La usa
Actualizar	La entidad	Nada	Edit (MVC) y PUT (API)
Eliminar	La llave	Nada	Delete (MVC) y DELETE (API)

Para mostrar nombres en lugar de números, «Listar todos» de Estudio y Teléfono debe incluir la persona y la profesión relacionadas.

5.4 Reglas de git

- main siempre compila; nadie sube directo a main después de la base del integrante 1.
- Una rama por integrante: feature/persistencia, feature/api-rest y feature/mvc.
- Se integra con pull request y otro integrante lo revisa.
- Si alguien necesita cambiar una interfaz DAO, lo avisa a los otros dos antes, porque rompe sus controladores.
- Commits pequeños con mensaje en español que diga qué se hizo («Agrega vistas CRUD de teléfono»).
- No se suben bin, obj ni .vs: el .gitignore va en el primer commit.
- El TAG se crea solo cuando main está probado de punta a punta por los tres.

6. Interfaz MVC (integrante 3)

Esta sección recoge la investigación y el plan del integrante 3: 4 controladores MVC y 20 vistas Razor para el CRUD de persona, profesión, estudios y teléfono.

6.1 Alcance y dependencias

Qué entrega

- PersonaController, ProfesionController, EstudioController y TelefonoController.
- Cinco vistas por entidad: Index (lista), Details (detalle), Create (crear), Edit (editar) y Delete (confirmar borrado).
- Listas desplegables en los formularios de Estudio (persona y profesión) y de Teléfono (dueño).
- Validaciones de formularios y mensajes de error claros.
- Menú del layout con las cuatro entidades y un enlace a Swagger.
- Capturas de cada vista para el documento final.

De qué depende

- Del integrante 1: las entidades generadas por scaffolding, las interfaces DAO y sus repositorios registrados en Program.cs.
- De las firmas de las interfaces acordadas por los tres. Si una firma cambia, los controladores MVC dejan de compilar.

Qué no le corresponde

- La base de datos, los scripts SQL ni los repositorios (integrante 1).
- Los controladores API, Swagger ni los endpoints /api/... (integrante 2).

6.2 Recorrido de una petición MVC

En MVC, el controlador recibe la petición, pide los datos al modelo y elige la vista que los muestra. El controlador y la vista son del integrante 3; el modelo y su acceso a datos, del integrante 1.

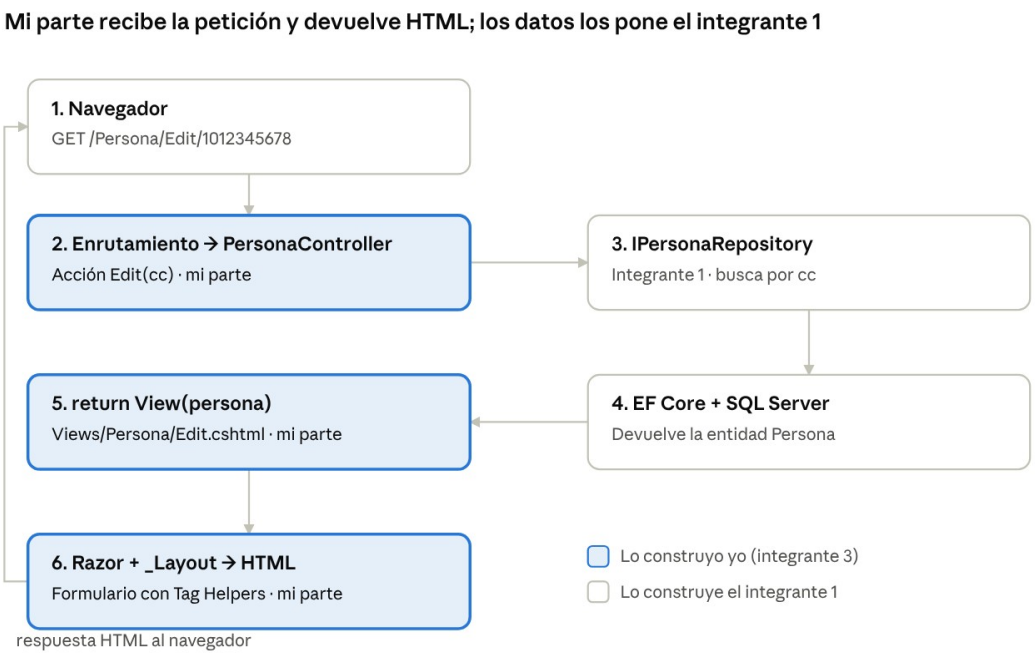


Figura 4. *Recorrido de una petición MVC en personapi-dotnet*

Al guardar el formulario el recorrido se repite con un POST: el controlador valida, llama a Actualizar del repositorio y redirige a la lista.

6.3 Controladores

Un controlador MVC es una clase que hereda de Controller; cada método público es una acción que responde a una URL y casi siempre devuelve una vista.

Enrutamiento

La plantilla MVC registra la ruta `{controller=Home}/{action=Index}/{id?}`. Así, /Persona/Edit/123 llama a la acción Edit de PersonaController con id = 123. Estudio tiene dos valores de llave y no cabe en {id}, por lo que se pasan como parámetros: /Estudio/Edit?idProf=3&ccPer=1012345678. El enlazador de modelos los asigna por nombre a los parámetros de la acción.

Tabla 16. *Resultados que devuelve una acción*

Resultado	Cuándo se usa
View(modelo)	Mostrar una vista con datos: Index, Details, formularios
RedirectToAction(nameof(Index))	Después de crear, editar o borrar con éxito
NotFound()	La llave no existe en Details, Edit o Delete
View(modelo) con errores	El formulario llegó con datos inválidos

Acciones asíncronas, GET y POST

Como los repositorios devuelven Task, las acciones son async y esperan cada llamada; mientras SQL Server responde, el hilo atiende otras peticiones. Create, Edit y Delete tienen dos versiones: la GET muestra el

formulario y la POST, marcada con [HttpPost], lo procesa. En Delete la POST se llama DeleteConfirmed y lleva [ActionName("Delete")] porque tendría la misma firma que la GET.

Patrón Post/Redirect/Get (PRG)

Después de un POST exitoso no se devuelve una vista sino una redirección a Index. Si el usuario recarga la página, el navegador repite el GET de Index y no el POST, de modo que no se crean registros duplicados. El mensaje «Persona creada» viaja en TempData, que sobrevive exactamente a una redirección.

6.4 Vistas Razor

Una vista es un archivo .cshtml que mezcla HTML con C# mediante la sintaxis Razor (@). Si PersonaController.Edit devuelve View() sin nombre, ASP.NET Core busca Views/Persona/Edit.cshtml y, si no existe, Views/Shared/Edit.cshtml.

Tabla 17. *Archivos compartidos de las vistas*

Archivo	Para qué sirve
Views/Shared/_Layout.cshtml	Plantilla común: encabezado, menú, pie y Bootstrap. Cada vista se dibuja dentro de RenderBody()
Views/_ViewStart.cshtml	Indica que todas las vistas usan _Layout
Views/_ViewImports.cshtml	Importa espacios de nombres y activa los Tag Helpers en todas las vistas
Views/Shared/_ValidationScriptsPartial.cshtml	Carga jQuery Validation para validar en el navegador
Views/Shared/Error.cshtml	Página de error genérica

Cada vista declara su modelo con @model: Index recibe una lista de Persona; Details, Create, Edit y Delete reciben una Persona. Así Visual Studio autocompleta las propiedades y un nombre mal escrito falla al compilar y no en tiempo de ejecución.

Tabla 18. *Tag Helpers usados en las vistas*

Tag Helper	Qué hace	Uso en las vistas
asp-controller, asp-action	Genera la URL de un enlace o formulario	Enlace «Editar» de cada fila
asp-route-{parámetro}	Agrega valores a la URL	asp-route-id en persona; asp-route-idProf y asp-route-ccPer en estudio
asp-for (input, label)	Genera id, name, tipo de campo, etiqueta y atributos de validación	Campos nombre, apellido, edad
asp-validation-for	Muestra el error de un campo	Debajo de cada campo
asp-validation-summary	Muestra errores generales del formulario	«La cédula ya existe»
select asp-for asp-items	Lista desplegable con la opción actual marcada	Persona y profesión en Estudio; dueño en Teléfono

El Tag Helper de formulario agrega por sí solo el token antifalsificación oculto. Las vistas parciales evitan repetir código (por ejemplo, _PersonaForm.cshtml compartido por Create y Edit), y Bootstrap 5, incluido en la plantilla, da estilo a tablas, botones y formularios sin CSS propio.

6.5 Formularios, validación y seguridad

Un formulario viaja en cuatro pasos: el navegador valida, el servidor enlaza los datos al modelo, valida de nuevo y solo entonces llama al repositorio.

- **Model binding:** ASP.NET Core toma los campos del formulario, la ruta y la cadena de consulta y los asigna por nombre al modelo. Un dato que no se puede convertir (por ejemplo «abc» en edad) queda como error en ModelState.
- **Validación en el servidor:** la acción revisa ModelState.IsValid antes de guardar. Si es falso, devuelve la misma vista con el modelo y los errores junto a cada campo.
- **Validación en el navegador:** los Tag Helpers generan atributos data-val-* y jQuery Validation Unobtrusive los aplica sin recargar la página. La del navegador es comodidad; la del servidor es la que protege.
- **Antifalsificación (CSRF):** cada acción POST lleva [ValidateAntiForgeryToken], que valida el token oculto del formulario.
- **Sobrepublicación (overposting):** [Bind] con la lista de campos permitidos en Create; en Edit se lee la entidad existente y se actualizan solo los campos permitidos (TryUpdateModelAsync) o se usa un ViewModel.
- **Errores de base de datos:** si guardar falla (por ejemplo, cédula repetida), la acción captura la excepción y agrega el mensaje con ModelState.AddModelError en lugar de mostrar una página de error.

Tabla 19. *Reglas de validación de los formularios*

Entidad	Campo	Regla
Persona	cc	Obligatoria, número positivo, no repetida; de solo lectura en Edit
Persona	nombre, apellido	Obligatorios, máximo 45 caracteres
Persona	genero	Obligatorio, solo M o F (lista de dos opciones)
Persona	edad	Opcional, entre 0 y 120
Profesion	id	Obligatorio, no repetido
Profesion	nom	Obligatorio, máximo 90 caracteres
Estudio	persona y profesión	Obligatorias, elegidas de lista; la pareja no puede repetirse
Estudio	univer	Opcional, máximo 50 caracteres
Estudio	fecha	Opcional, no futura
Telefono	num	Obligatorio, máximo 15 caracteres, no repetido
Telefono	oper	Obligatorio, máximo 45 caracteres
Telefono	dueño	Obligatorio, elegido de lista

Las entidades generadas por scaffolding no traen DataAnnotations; las reglas se ponen en clases de metadatos o en ViewModels para no editar los archivos generados, que se sobrescriben si se vuelve a ejecutar Scaffold-DbContext. Con .NET 10 los tipos de referencia anulables están activos, así que un string no anulable ya se trata como obligatorio.

6.6 Casos especiales del modelo

- **Llave compuesta de Estudio:** se identifica por la pareja (idProf, ccPer). Todos los enlaces llevan los dos valores. En Create se eligen de listas; en Edit se muestran de solo lectura, porque cambiarlas sería cambiar de registro. Si la pareja ya existe se muestra «Esta persona ya tiene registrada esa profesión».
- **Listas desplegables:** Estudio necesita personas (valor cc, texto nombre y apellido) y profesiones (valor id, texto nom); Teléfono necesita personas. Se pasan en EstudioFormViewModel y TelefonoFormViewModel, como recomienda Microsoft en lugar de ViewBag. Si el formulario vuelve con errores, las listas se recargan antes de devolver la vista.
- **Género:** se elige de una lista de dos opciones (Masculino → M, Femenino → F) y en las vistas se muestra el texto completo.
- **Detalle de Persona:** muestra sus datos y, debajo, sus estudios y teléfonos; es la vista que mejor demuestra las relaciones del modelo.
- **Borrados con dependientes:** con CASCADE, borrar una persona o una profesión borra sus estudios (y teléfonos, en el caso de persona). La confirmación de Delete lo advierte y muestra cuántos registros dependientes hay.
- **Nombres en lugar de llaves:** Index de Estudio muestra «Ana Pérez — Ingeniería de Sistemas» en lugar de «1012345678 — 3».

6.7 Inventario de controladores y vistas

Tabla 20. *Inventario: 4 controladores, 32 acciones y 20 vistas*

Controlador	Acción	Verbo	Ruta	Vista	Método del repositorio
PersonaController	Index	GET	/Persona	Index	Listar todos
PersonaController	Details	GET	/Persona/Details/{cc}	Details	Buscar por cc, con estudios y teléfonos
PersonaController	Create	GET	/Persona/Create	Create	—
PersonaController	Create	POST	/Persona/Create	Redirige a Index	Crear
PersonaController	Edit	GET	/Persona/Edit/{cc}	Edit	Buscar por cc
PersonaController	Edit	POST	/Persona/Edit/{cc}	Redirige a Index	Actualizar
PersonaController	Delete	GET	/Persona/Delete/{cc}	Delete	Buscar por cc
PersonaController	DeleteConfirmed	POST	/Persona/Delete/{cc}	Redirige a Index	Eliminar
ProfesionController	Las mismas 8	GET y POST	/Profesion/…/{id}	Las 5 vistas	Las 5 operaciones
EstudioController	Las mismas 8	GET y POST	/Estudio/…?idProf=&ccPer=	Las 5 vistas	Las 5 operaciones + listar personas y profesiones
TelefonoController	Las mismas 8	GET y POST	/Telefono/…/{num}	Las 5 vistas	Las 5 operaciones + listar personas

6.8 Plan de trabajo

1. Reunión de los tres para acordar arquitectura y firmas DAO.
2. Preparar el ambiente: Visual Studio 2026, SQL Server 2025 Express, SSMS 22 y el repositorio clonado.
3. Cuando el integrante 1 suba la base a main, crear la rama feature/mvc.
4. Verificar que la aplicación arranque y que persona_db tenga datos.
5. PersonaController y sus 5 vistas. Atajo: Agregar > Controlador > «Controlador de MVC con vistas que usa Entity Framework». Ese controlador usa el DbContext directo y rompe el patrón DAO, así que se reemplaza por IPersonaRepository conservando las vistas.
6. Ajustar las vistas de Persona: género como lista, cédula de solo lectura en Edit, textos en español, estudios y teléfonos en Details.
7. ProfesionController y sus 5 vistas.
8. EstudioController: ViewModel con listas, llave compuesta en rutas y enlaces, mensaje de pareja repetida.
9. TelefonoController: ViewModel con lista de dueños.
10. Menú en _Layout y página de inicio.
11. Validaciones, mensajes y plan de pruebas.
12. Pull request de feature/mvc a main, revisado por un compañero.
13. Pruebas finales con los tres, capturas de las vistas y aporte a las conclusiones.

6.9 Plan de pruebas de la interfaz

Tabla 21. *Casos de prueba de la interfaz MVC*

#	Caso	Pasos	Resultado esperado
1	Listar	Abrir /Persona	Tabla con los datos del DML y enlaces Detalles, Editar, Borrar
2	Crear válido	Llenar el formulario y guardar	Redirige a la lista con mensaje de éxito
3	Crear vacío	Guardar sin llenar campos	Errores junto a cada campo, sin recargar
4	Crear sin JavaScript	Desactivar JavaScript y guardar vacío	El servidor devuelve los mismos errores
5	Llave repetida	Crear persona con cédula existente	«La cédula ya existe»; no se pierde lo escrito
6	Longitud máxima	46 caracteres en nombre	Error de longitud
7	Editar	Cambiar el apellido y guardar	La lista muestra el cambio; la cédula no se edita
8	No existe	Abrir /Persona/Details/999	Página 404, no error de servidor
9	Borrar	Borrar una persona con estudios	La confirmación avisa los dependientes
10	Recargar tras guardar	Crear y presionar F5 en la lista	No se crea un duplicado (PRG)
11	Listas (Estudio, Teléfono)	Abrir Create	Listas con nombres
12	Error con listas	Guardar con un error	El formulario vuelve con las listas llenas
13	Pareja repetida (Estudio)	Crear un estudio existente	Mensaje de pareja repetida
14	Coherencia con la API	Crear en la vista y consultar en Swagger	El registro aparece en GET /api/personas

#	Caso	Pasos	Resultado esperado
15	Navegación	Usar todos los enlaces del menú	Ningún enlace roto

7. Procedimiento

Se sigue el orden del enunciado, agrupado en seis fases. Lo marcado como «agregado» no está en el enunciado, pero hace falta para cumplir el stack.

7.1 Fase 1 — Repositorio

1. El repositorio público personapi-dotnet existe en la cuenta de Sara Muñoz; ella agrega al resto del equipo como colaboradores.
2. Cada integrante clona el repositorio en una carpeta local (git clone).

7.2 Fase 2 — Ambiente

1. Instalar SQL Server 2025 Express con la instalación Básica; queda la instancia localhost\SQLEXPRESS.
2. Instalar SQL Server Management Studio 22.
3. Instalar Visual Studio Community 2026 con las cargas de trabajo de la sección 3.1 y verificar el SDK de .NET 10.

7.3 Fase 3 — Base de datos

1. En SSMS, crear la base persona_db y asignarle como propietario al usuario sa.
2. Crear las cuatro tablas según el modelo de datos (sección 4.4); ese script es el DDL de la entrega.
3. Insertar datos de prueba en orden profesion, persona, estudios, telefono; ese script es el DML.
4. Guardar ambos scripts en la carpeta scripts del repositorio.

7.4 Fase 4 — Proyecto y persistencia

1. Crear el proyecto con la plantilla Aplicación web de ASP.NET Core (Modelo-Vista-Controlador), llamado personapi-dotnet, .NET 10.0, sin autenticación y sin HTTPS, dentro de la carpeta clonada.
2. Activar Ver > Explorador de objetos de SQL Server, agregar la conexión local Express y probarla.
3. Instalar por NuGet, en la rama 10.0.x: Microsoft.EntityFrameworkCore, Microsoft.EntityFrameworkCore.SqlServer y Microsoft.EntityFrameworkCore.Tools.
4. Agregado: instalar Swashbuckle.AspNetCore 10.x para Swagger.
5. Crear la carpeta Models/Entities y ejecutar Scaffold-DbContext en la Consola del Administrador de paquetes (comando a continuación).
6. Mover la cadena de conexión a appsettings.json y quitar la que el scaffolding deja escrita dentro del contexto.

```
Scaffold-DbContext "Server=localhost\
SQLEXPRESS;Database=persona_db;Trusted_Connection=True;TrustServerCertificate=true"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models/Entities
```

7.5 Fase 5 — Capas de la aplicación

1. Crear la carpeta Interfaces con una interfaz por entidad y sus cinco operaciones.

2. Crear la carpeta Repository con las clases que implementan esas interfaces usando PersonaDbContext.
3. Agregado: registrar en Program.cs el contexto, los repositorios y Swagger.
4. Crear los controladores API en Controllers/Api y los controladores MVC con sus vistas.
5. Probar cada endpoint en Swagger y cada vista en el navegador; tomar capturas.

7.6 Fase 6 — Despliegue y entrega

1. Desplegar: ejecutar desde Visual Studio (perfil Kestrel) o publicar con Compilar > Publicar a una carpeta y ejecutar el .dll publicado. Documentar el puerto y la URL de Swagger.
2. Escribir el README: requisitos, creación de la base, cadena de conexión, compilación, ejecución y URLs.
3. Hacer push de todo a main.
4. Crear el TAG anotado de entrega (por ejemplo v1.0.0) y subirlo; en GitHub puede convertirse en Release.
5. Entregar la URL del TAG y este documento.

8. Lista de verificación de la entrega

Cada requisito del enunciado, con su responsable. Se marca cuando está verificado por los tres.

Tabla 22. *Lista de verificación de la entrega*

Requisito del enunciado	Responsable	Verificado
Base persona_db con dueño sa y tablas según el modelo	Integrante 1	☐
Script DDL y script DML que corren sin errores en una base vacía	Integrante 1	☐
Entidades y contexto generados con Scaffold-DbContext	Integrante 1	☐
Cadena de conexión en appsettings.json	Integrante 1	☐
Interfaces y repositorios con el CRUD	Integrante 1	☐
Endpoints REST del CRUD probados en Swagger	Integrante 2	☐
Vistas y controladores MVC del CRUD probados en navegador	Integrante 3	☐
Aplicación desplegada	Los tres	☐
README con configuración, ambiente, compilación y despliegue	Los tres	☐
Push a main y TAG creado; URL verificada en modo incógnito	Los tres	☐
Documento con portada, marco conceptual, diseño, procedimiento, conclusiones y referencias, con capturas	Los tres	☐
Cambio de versiones confirmado con el profesor	Los tres	☐

9. Conclusiones y lecciones aprendidas

Borrador para ajustar en equipo cuando el laboratorio esté implementado y probado.

9.1 Conclusiones

- Un monolito en capas permite entregar vistas y API REST desde un solo despliegue sin mezclar responsabilidades: los controladores no tocan la base de datos directamente.
- El patrón DAO, implementado como interfaz más repositorio, permite que el controlador MVC y el controlador API reutilicen exactamente la misma lógica de acceso a datos.
- El enfoque database-first con scaffolding acelera el arranque, pero obliga a que el esquema esté bien definido antes de escribir código.
- Swagger convierte la documentación de la API en algo que se prueba, no solo que se lee.
- Repartir el trabajo por capas funciona si las interfaces DAO se acuerdan primero: son el contrato que permite trabajar en paralelo.

9.2 Lecciones aprendidas

- Leer el enunciado completo antes de instalar: el stack y el procedimiento piden versiones distintas, y ambas ya están sin soporte.
- Alinear la versión de los paquetes NuGet con el framework: con .NET 10, EF Core va en la rama 10.0.x.
- Desde .NET 9 Swagger no viene en las plantillas; hay que agregarlo y fijar la versión de OpenAPI que se entrega.
- Traducir el modelo de MySQL a SQL Server con cuidado: ENUM, TEXT y el ancho de los INT no existen igual.
- Las llaves compuestas complican rutas, formularios y búsquedas; conviene resolverlas primero.
- Definir el comportamiento al borrar (CASCADE o borrado manual) antes de programar el DELETE de persona.
- Un repositorio vacío no se puede abrir en modo remoto: hay que clonarlo y hacer un primer commit para que exista la rama main.

10. Referencias

Organizadas por fuente.

10.1 Material de la guía del curso

- hdeleon.net. (s. f.). Video 1 de la guía [Video]. YouTube.
- hdeleon.net. (s. f.). Video 2 de la guía [Video]. YouTube.
- Guamán, V. (s. f.). API con ASP.NET MVC 6 y SQL Server mediante Entity Framework Core 6 code-first, parte 1. dev.to.
- Guamán, V. (s. f.). API con ASP.NET MVC 6 y SQL Server mediante Entity Framework Core 6 code-first, parte 2. dev.to.
- C# Corner. (s. f.). Building ASP.NET Web API in .NET Core with Entity Framework.
- Microsoft. (s. f.). Entity Data Model. Microsoft Learn.

10.2 Documentación oficial de Microsoft (ASP.NET Core 10.0)

- Microsoft. (s. f.). Información general de ASP.NET Core MVC. Microsoft Learn.
- Microsoft. (s. f.). Enrutamiento a acciones de controlador. Microsoft Learn.
- Microsoft. (s. f.). Vistas en ASP.NET Core MVC. Microsoft Learn.
- Microsoft. (s. f.). Diseño (layout) en ASP.NET Core. Microsoft Learn.
- Microsoft. (s. f.). Tag Helpers en formularios. Microsoft Learn.
- Microsoft. (s. f.). Enlace de modelos. Microsoft Learn.
- Microsoft. (s. f.). Validación de modelos. Microsoft Learn.
- Microsoft. (s. f.). Prevención de ataques de falsificación de solicitudes (CSRF). Microsoft Learn.
- Microsoft. (s. f.). Inserción de dependencias en ASP.NET Core. Microsoft Learn.
- Microsoft. (s. f.). Tutorial: CRUD con EF Core en una aplicación MVC. Microsoft Learn.
- Microsoft. (s. f.). Scaffolding (ingeniería inversa) en EF Core. Microsoft Learn.
- Microsoft. (s. f.). Arquitecturas comunes de aplicaciones web. En Architect Modern Web Applications with ASP.NET Core and Azure.

10.3 Herramientas y versiones

- Microsoft. (2026). Descargas y estado de soporte de .NET. Consultado el 20 de octubre de 2026.
- Microsoft. (2026). Notas de versión de SQL Server Management Studio 22. Microsoft Learn.
- MSSQLTips. (s. f.). SQL Server 2025 Express Edition: descarga, instalación y configuración.
- Domaindrivendev. (s. f.). Swashbuckle.AspNetCore. GitHub.
- Murugan, M. (s. f.). Swagger alternatives and OpenAPI in .NET. codewithmukesh.

10.4 Especificaciones, patrones y arquitectura

- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures, cap. 5: REST [Tesis doctoral]. Universidad de California, Irvine.
- OpenAPI Initiative. (2020). OpenAPI Specification 3.0.3.

- Fowler, M. (s. f.). Repository. Catalog of Patterns of Enterprise Application Architecture.
- Fowler, M. (2015). MonolithFirst.
- Oracle. (s. f.). Core J2EE Patterns: Data Access Object.
- Smith, S. (2016). Feature slices for ASP.NET Core MVC. MSDN Magazine.

10.5 Proyectos de referencia

- NimblePros. (2026). eShopOnWeb: aplicación de referencia ASP.NET Core. GitHub.
- Smith, S. (2026). Clean Architecture Solution Template. GitHub.